ENG2040 Assignment 1: Marketplace Orders to a Model-Ready Dataset

Noel Thomas (25011152)

October 2026

Summary

- One command (`uv run python run.py`) rebuilds every output from `raw.sql`. A fresh clone gives byte-identical files.
- All 20,000 orders are kept and accounted for: 16,482 get a reliable label, the rest carry a reason why not.
- 6.7% of labelled orders arrive late (95% interval 6.3–7.1%). Late orders come in waves, short estimates are late twice as often, and RJ has the most late deliveries above the average.
- The feature table has 16,482 rows and 72 numeric columns, built only from what is known at approval. Leakage checks pass.
- Every cleaning fix is backed by an independent record (a *witness*). Without one, the value becomes missing instead of being guessed.

Data: Olist e-commerce extract, modified for the course [9]. Code: `pipeline.ipynb` (one section per step), `sql/`, `bench.py`, `run.py`. The README maps every step to its inputs, outputs and row meaning, and every brief question to where it is answered.

1 Loading the data

Tables and keys. The extract has 8 tables. *Orders* is the grain: one row per order.

table	rows	key	one row is
orders	20,000	order_id	an order
customers	20,000	customer_id	the customer record of one order
order_items	22,662	order_id + order_item_id	one item (2 of a product = 2 rows)
order_payments	20,875	order_id + payment_sequential	one payment, in centavos
order_reviews	19,966	review_id + order_id	one review
products / sellers	11,334 / 2,071	product_id / seller_id	catalogue rows, linked from items

Assembling one row per order.

- Items, payments and reviews are one-to-many. Joining them raw multiplies rows (3 items × 2 payments = 6 rows) and double counts totals.
- So `sql/orders.sql` first aggregates each of them per order inside SQLite, then left joins the aggregates onto orders. No fan-out, and no order is lost.
- Sums skip NULLs, so a NULL count sits next to every sum. Payments stay in integer centavos, so totals are exact.
- The query plan shows each GROUP BY walking the `order_id` index, and each aggregate joined back by index lookup.

Relationships and totals preserved (`key_links.csv`, `checks.csv`):

- 0 items, payments or reviews point to a missing order, so nothing is dropped when they roll up.
- The other direction has gaps: 769 orders have no customer row, 142 have no items, 910 items have no product row and 893 have no seller row. These stay as NULLs. Dropping is a cleaning decision.
- 0 duplicate orders and 0 customer records shared by two orders. The 998 repeated item rows are how quantity is stored.

- Order count, item count, price total (54,446,205.77), payment total (280,524,473 centavos) and review count are equal in the database and in the output.

Why this approach is the most efficient choice. The same table was built 8 ways at 1×, 3× and 10× the data. Each run is a fresh process; values are the median of 3, with a baseline subtracted (`bench.py`).

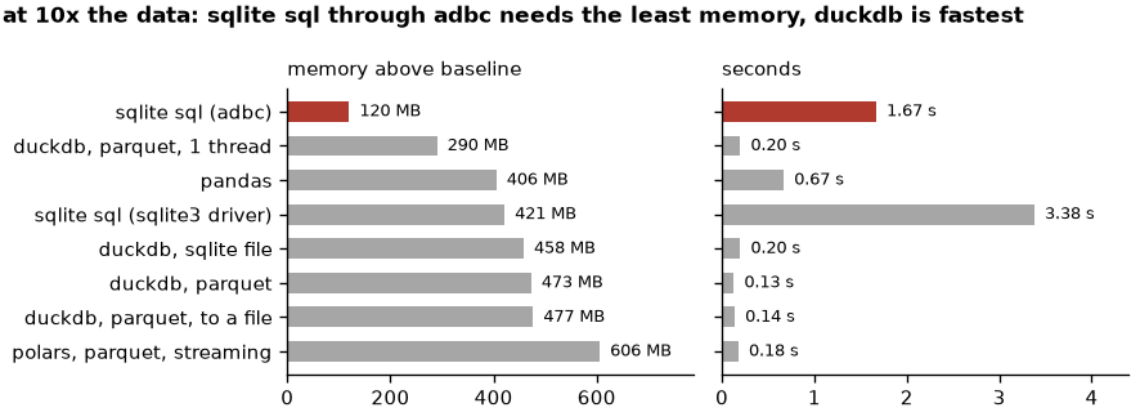


Figure 1: At 10× the data, SQLite SQL through ADBC needs the least memory. DuckDB is fastest but needs 2.4–4× the memory.

- **My first estimate was wrong.** I expected SQL to need half of pandas' memory. With Python's `sqlite3` driver it needed the same (421 vs 406 MB), because the driver builds a Python object per value: 20 MB of objects for a result that is 7 MB as a dataframe. ADBC hands over Arrow columns instead [3], and then SQL needs under a third of pandas' memory (120 vs 406 MB).
- **DuckDB** [4] is 5–13× faster but holds 290–480 MB. Its hash tables for the joins cost more than the result. Writing the result to a file did not help, and a memory cap made the query fail.
- **Polars** (lab 7) ran the same SQL file lazily with its streaming engine [5]. It needed the most memory (606 MB). Streaming helps when the inputs do not fit in memory; here they do.
- **Decision:** no engine wins on both. Memory decides whether a step fits on a machine at all, and 1.7 s at 10× does not matter for a batch step. So SQLite through ADBC. If runtime mattered more, DuckDB would win.
- **Storage:** Parquet keeps types (timestamps, booleans, NULLs) that CSV turns into text, and is smaller (3.29 vs 5.25 MB for orders) [6].

2 Data quality and cleaning

Rule for every problem, strongest first. No order is deleted.

1. **Fix** only when an independent record (a witness) shows the true value.
2. **Set missing** when the value is wrong but nothing shows the true one.
3. **Flag** when the field feeds neither labels nor features.

Every changed value is logged with its key, column, rule, old and new value (`changes.parquet`).

problem	action	evidence
price < 0 (408), freight < 0 (458)	absolute value	price + freight then matches the payment for 87% / 90% of orders, the same as clean orders (90%)
price $>$ 5,000 (417), freight $>$ 500 (473)	divide by 1,000	matches payments for 94% / 88%. Dividing by 100 matches 0%. The same product sells elsewhere at the fixed price for 178 of 277
sizes $>$ 200 cm, weight $>$ 100 kg	missing	freight says impossible: "over 100 kg" items ship for R$15.2, like 0.4–1.4 kg items (R$16.1), not 20–30 kg ones (R$74.6). Freight cannot give the exact value
-1 photos, name or description length	missing	-1 is the only negative value these ever take: a placeholder
state ZZ or blank	missing, then recovered from zip prefix (895 customers, 1,032 sellers)	a zip prefix sits in one state. 14 prefixes with two states are not used. Recovered rows are flagged
impossible dates (2018-02-30), sizes ≤ 0, `unmapped_category`	missing	the value cannot exist
status with spaces or capitals	strip, lowercase	formatting only

Labels. `delivered_after_estimate` = 1 if the delivery calendar date is after the estimated date, 0 if on or before it (local clock). An order is labelled only if it is a usable completed delivery. The first rule that applies gives the reason it stays unlabelled.

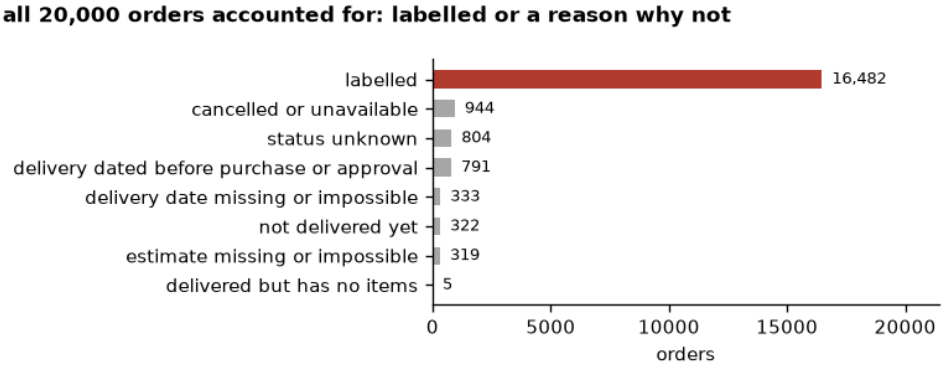


Figure 2: Every order is either labelled or has one reason why not.

Checked against payments again. After cleaning, price + freight matches the payment for 16,302 of 18,122 complete orders (14,845 before). Of the 1,820 that still do not match, 919 have a negative payment and 832 paid exactly R$7.23 more.

Unresolved, and what it affects.

- **R$7.23 extra** on 832 orders: same payment types, states and months as all orders (credit card 76% vs 72%, SP 40% vs 42%), never a payment row of its own. Spread evenly, so it cannot bias a comparison. Left as is.
- **Payments** (negative values, 929 instalments of -1) are left alone: they are never predictors or labels, only a witness. 0 instalments is valid.
- **Carrier pickup before approval** (270): flagged. The carrier date is in neither label nor features.
- **`not_reported`** (804) and the deliveries dated exactly 7 days before approval (782 of 791 too early): unlabelled, since a fix would be a guess. Section 3 tests putting them back.
- **Extreme delays** (up to 188 days, 14 orders over 60): kept. Their dates are consistent and nothing says they are wrong.
- A missing optional catalogue field never removes an order (brief).

3 Exploratory analysis

Rates come with 95% Wilson intervals. The textbook $p \pm 1.96\,\mathrm{se}$ interval fails for small groups (0 late out of 8 gives 0–0%); Wilson gives 0–32% and is the recommended choice for proportions [1, 2].

Q1. How often are orders late, and by how much?

- 6.7% of labelled orders are late (6.3–7.1%), about 1 in 15.
- A late order is typically a week late (median 7 days, middle half 3–13), and 1 in 8 is 3+ weeks late.
- The typical on-time order arrives 13 days early, so estimates are padded.

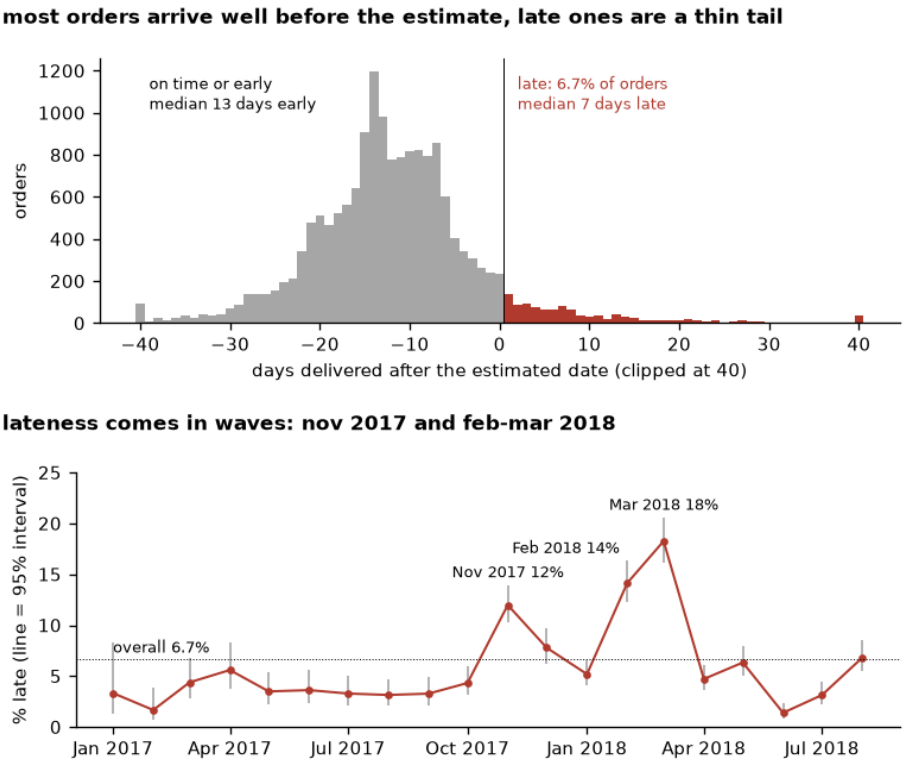


Figure 3: Top: most orders arrive well before the estimate. Bottom: lateness comes in waves, with 18% of March 2018 orders late against 1–8% in normal months.

Q2. How does time from approval to the estimate relate to lateness?

- Estimates under a week are late 16% of the time, twice any other group. From 1 to 5 weeks it is 5–7%; beyond 6 weeks, under 2%.
- **Is it really state or season?** No short estimate falls in the spike months. Within the same state and month, short estimates are late 16.3% vs 4.2%.
- This is an association: seller, product or carrier could sit behind it. A longer estimate would not make deliveries faster, but a short estimate is a useful risk signal at approval.

Q3. Which states, and where should operations look first?

- States are ranked by **excess late orders** = late orders − orders × overall rate. This counts the late deliveries a state adds. A high rate on 9 orders is mostly noise; a slightly raised rate on 2,000 orders is many late deliveries.
- **RJ first** (+98), then BA (+26) and CE (+22). RJ's rate (11.5%) is not the highest, but it is on 2,036 orders with a tight interval.

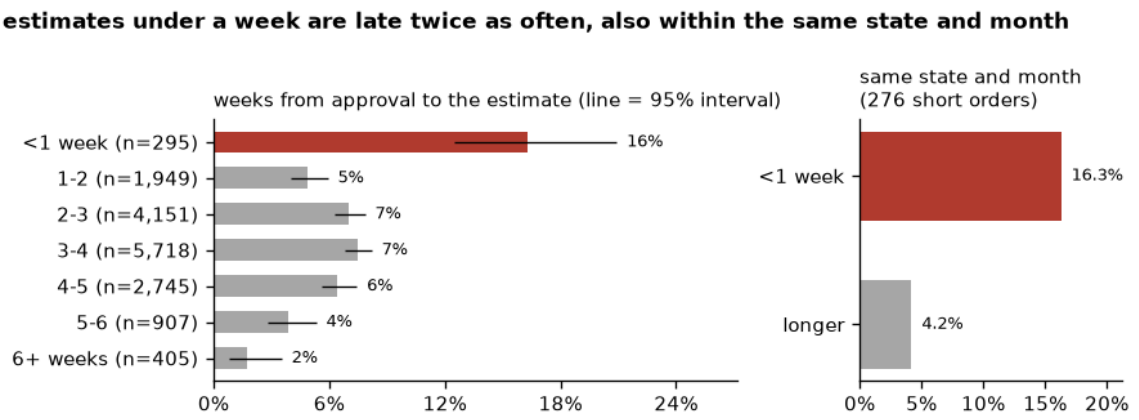


Figure 4: Short estimates are late twice as often, also when compared within the same state and month.

- **Drill-down:** 65 of RJ's excess falls in the spike months and 94 is on sellers in SP. So start with the SP → RJ route at peak times. BA's excess is mostly outside the spikes: a steady problem.
- RR (9 orders) and AP (8) look extreme but are too small. By chance 1–2 of 27 states would sit outside the overall rate; 12 do, so states really differ.
- 951 labelled orders (6%) have no state, 741 of them because the customer row is missing. They are left out of the ranking.

Data quality, sensitivity and confidence

- Each cleaning choice was changed one at a time (labelling `not_reported`, undoing the 7-day shift, dropping recovered states, dropping orders with missing prices). The late rate stays at 6.4–6.8%, and the top 3 states are always RJ, BA and CE.
- The real gap is coverage: 18% of orders have no label. The 944 cancelled orders could move the late rate from 6.4% (none late) to 11.8% (all late). 743 of them were already with the carrier, so 6.7% is best read as a lower bound.
- This is one modified extract (2016–2018). RJ, BA, CE, the spike months and short estimates are where to look first, not proven causes.

4 Feature engineering

Rows and inputs. One row per labelled order (16,482). Columns that are not available at approval (payments, status, carrier and delivery dates, reviews, `shipping_limit_date`) are dropped *before* any feature is built, so using one by mistake fails. IDs and labels are in a separate file, `ids_labels.parquet`, in the same row order.

group	features	columns
timing	days from approval to estimate, hours from purchase to approval, purchase month / weekday / hour as sin + cos	8
basket	items, products, sellers, price total, freight total, freight share	6
product / seller	total weight, total volume, mean photos, seller states, share of items from the customer's state	5
missing flags	one per feature that can be missing	8
customer state	one-hot, 27 states + missing	28
category	most expensive item's category, top 15 + other + missing	17

Representation choices.

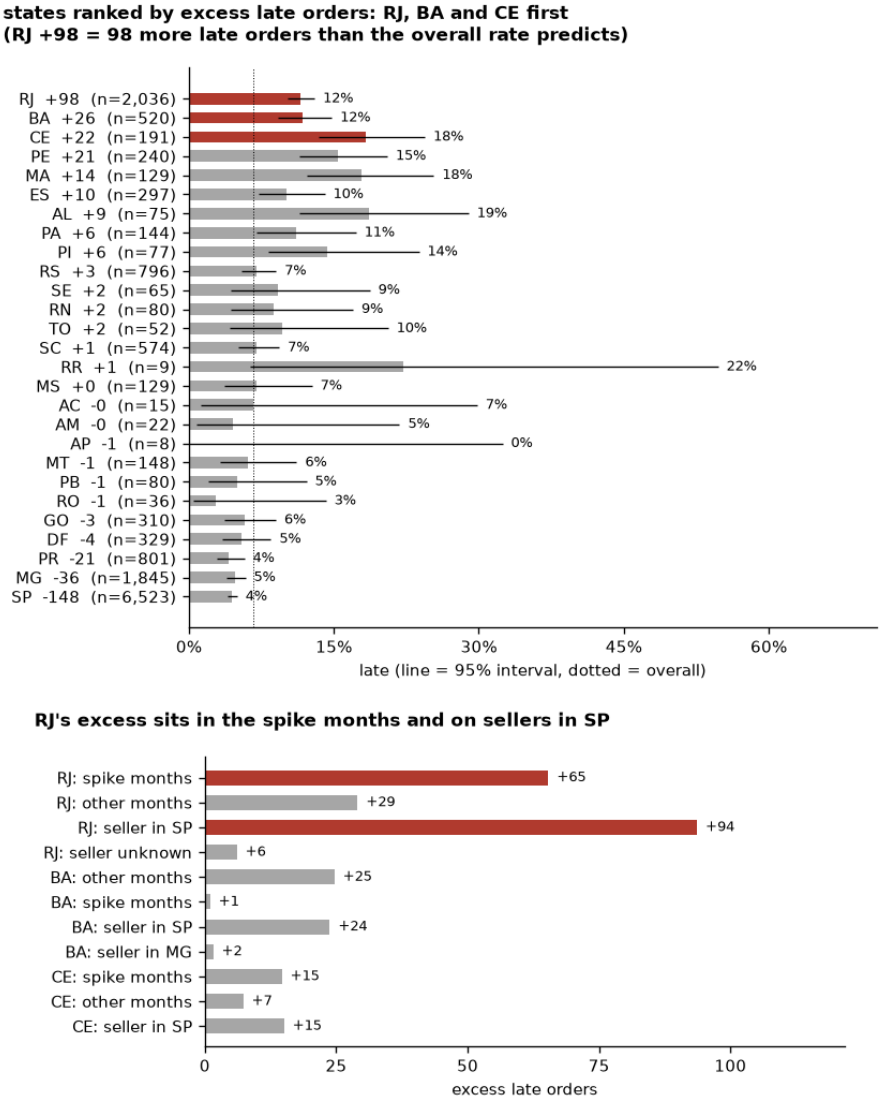


Figure 5: Top: states ranked by excess late orders. Bottom: RJ's excess sits in the spike months and on sellers in SP.

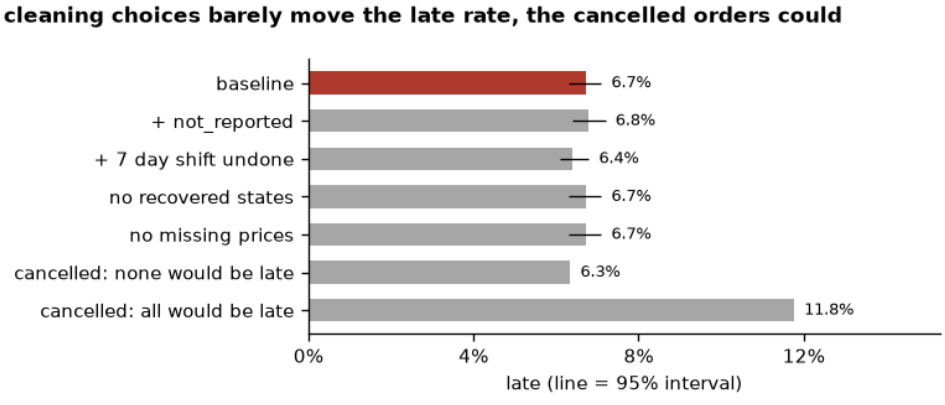


Figure 6: Cleaning choices barely move the late rate; the unlabelled cancelled orders could move it far more.

- Numbers stay numbers. Sums are per order, because the label is per order. A sum with a missing part is set to missing, since it would be too low.
- Missing stays NaN with a `_missing` flag. Imputing here would use statistics from future test orders; it belongs inside model fitting.
- Cyclic time (month, weekday, hour) is encoded as sin + cos, so December sits next to January. Year is not used, because a model runs on years it has never seen.
- Rare categories are grouped as "other"; otherwise they would be near-empty columns.

Which features are kept. Two were dropped as copies: `est_days_from_purchase` (= lead days + approval hours / 24) and `n_categories` (= not category missing). The highest remaining correlation is 0.78 (weight vs volume). Selecting features by how well they predict would let test orders choose the features, so it is left to model fitting. `feature_descriptions.csv` gives each feature's meaning and source columns.

Leakage evidence.

- No feature's source column is on the forbidden list (checked in code).
- Each feature alone scores an AUC of 0.43–0.61. The outcome `days_late`, scored as a control, gets 1.0, which shows the check would catch a leak.
- Extra: a gradient-boosted tree [8] trained on orders before April 2018 and tested on later ones scores a test AUC of 0.611. Lead time alone scores 0.534, and the leak control 1.000. So there is modest signal and no leak. Lateness comes in waves that a model trained on the past cannot foresee.

5 Reproducibility and resources

One command. `run.py` checks the supplied files against SHA-256 hashes, rebuilds the database with the supplied loader, runs the engine comparison and the notebook, then writes a hash of every output (`manifest.sha256`). Two runs, and a fresh clone, give identical hashes. `run.py -fast` reuses the engine comparison. Environment: Python 3.12, pandas 3.0, pyarrow 26, pinned in `uv.lock` (`environment.json`).

Estimated vs measured. Memory is this process's peak RSS above the step's start, sampled every 10 ms. Storage is the files a step writes, which are also the next step's input. Estimate rules, from the in-memory size of each step's data: memory 2–3×; Parquet 0.3–0.5× of the tables written; under 1 s per step except charts.

step	data MB	memory est.	measured (6 runs)	saved est.	measured	seconds
0 setup db	18.3	33–52	205 (whole process)	18–27	24.4	< 5
1 load	12.9	26–39	47–78	3.9–6.4	5.2	0.2
2 clean	17.6	35–53	59–68	5.3–8.8	6.6	0.5
3 analysis	21.4	43–64	59–74	< 1	0.25	1.7
4 features	24.0	48–72	52–62	3.2–5.3	1.2	0.2

- Saved storage lands inside the estimate for steps 0–3, and every step meets its runtime estimate. Temporary storage is 298 MB, only for the 10× engine comparison, and is deleted afterwards.
- Memory: step 4 lands inside its estimate and step 3 at or above the top of its range, while steps 1 and 2 need about 3.5–6× their data. The likely cause, not measured separately, is Arrow buffers held before pandas wraps them, and freed memory not returned to the OS. RSS moves between runs (up to about 30 MB), so the table shows the range over 6 runs.
- Step 4 saved 1.2 MB against 3.2–5.3: most feature columns are one-hot zeros, which Parquet compresses far better than the rule assumes.
- **Step 0 was the big miss** (205 vs 33–52 MB). The reviews contain emoji, and one character above U+FFFF makes Python store the whole 18-million-character text at 4 bytes per character (73 MB) [7], plus the raw bytes and decoding copies. Measured by hand: reading as bytes peaks at

33 MB, decoding at 123 MB, and reading as text (what `setup_db.py` does) at 196 MB. The loader is supplied, so it stays.

Limitations. Findings describe this modified extract; associations are not causes. 18% of orders are unlabelled, and the cancelled ones make the late rate a lower bound. The memory estimate gaps are explained but not measured separately.

Use of AI. I used Claude (Anthropic) as an assistant: to suggest and test other engines (DuckDB, Polars) I had not used before, to make the code simpler, to typeset this report in LaTeX, and to check that every number in the text matches the outputs. I made the decisions and checked the results, and I can explain each of them.

References

[1] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[2] L. D. Brown, T. T. Cai, A. DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2):101–133, 2001.

[3] Apache Arrow. ADBC: Arrow Database Connectivity. https://arrow.apache.org/adbc/

[4] M. Raasveldt, H. Mühleisen. DuckDB: an embeddable analytical database. *SIGMOD*, 2019.

[5] Polars user guide: streaming. https://docs.pola.rs/

[6] Apache Parquet documentation. https://parquet.apache.org/docs/

[7] M. v. Löwis. PEP 393: Flexible string representation. 2010. https://peps.python.org/pep-0393/

[8] F. Pedregosa et al. Scikit-learn: machine learning in Python. *JMLR*, 12:2825–2830, 2011.

[9] Olist. Brazilian E-Commerce Public Dataset (modified for this course). CC BY-NC-SA 4.0. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce